

Test Architecture — Frontend + Backend

What we test across the two repos behind stipes.tech, **how** each thing is tested and why that way, and **when** to reach for the Claude skills that generate most of it.

Frontend	<code>stipes-nextjs-tailwind</code> — Next.js 14 App Router, Vitest + Playwright
Backend	<code>stipes-openai-chat</code> — Go 1.22, stdlib <code>testing</code> only
Deployed	<code>https://www.stipes.tech</code> → <code>https://stipes-openai-chat.vercel.app</code>

Companion documents: `CRITICAL_FLOWS.md` (frontend invariants) and `docs/TESTING_STRATEGY.md` in the backend repo (Go layer detail).

Start here: new to the codebase → §1–2. Writing a test → §3–5. Want the tooling to do it → §6.

Read this on GitHub or in an editor with Mermaid support — the diagrams are the point. The PDF renders them as source, not pictures. This document describes structure, not status. Counts and pass rates live in the test run, not here — the one place they appear is the coverage thresholds, which are enforced.

1. System shape

Two independently deployed services. The browser never talks to the Go service directly — the Next.js route handler proxies, which is what supplies CORS and keeps the OpenAI key out of the client.

```

flowchart LR
    subgraph Browser
        CW["ChatWidget / ChatWindow<br/>(client component)"]
    end

    subgraph FE["Frontend – Vercel (Next.js 14)"]
        RSC["Server components<br/>page.tsx, blog/[slug]"]
        MDX["lib/mdx.ts<br/>filesystem MDX"]
        API["/api/chat<br/>route handler (proxy)"]
    end

    subgraph BE["Backend – Vercel (Go serverless)"]
        H["api/chat.go<br/>Handler"]
        PK["pkg/openai<br/>prompt policy"]
        EM["pkg/embedded<br/>resume + notes"]
    end

    OAI["OpenAI Responses API"]

    CW -- "POST {message}" --> API
    API -- "POST {message}" --> H
    H --> PK --> OAI
    EM -.context.-> PK
    RSC --> MDX

    classDef fe fill:#1e3a5f,stroke:#4a90d9,color:#fff
    classDef be fill:#1e4620,stroke:#5cb85c,color:#fff
    classDef ext fill:#4a3520,stroke:#d9a34a,color:#fff
    class RSC,MDX,API,CW fe
    class H,PK,EM be
    class OAI ext

```

Why the proxy exists: the Go service sends no CORS headers at all (`grep -rn "Access-Control"` finds nothing), so a browser could not call it cross-origin even if we wanted it to. `/api/chat` is the boundary.

2. What we test, and how

Both repos split tests by **cost and determinism**, not by the usual unit/integration taxonomy. The question each layer answers is different, so a failure at each layer means something different.

```

flowchart TB
    subgraph L4["Layer 4 – Live · nondeterministic · deliberate"]
        SM["FE: @smoke vs deployed site<br/>'is production actually up?'"']
        EV["BE: -tags eval corpus + judge<br/>'does the model still answer well?'"']
    end
    subgraph L3["Layer 3 – Contract · mocked · free"]
        CE["FE: chat-errors specs<br/>'do we handle every response shape?'"']
        CT["BE: contract_test.go<br/>'is the wire format unchanged?'"']
    end
    subgraph L2["Layer 2 – Component · offline · free"]
        E2E["FE: Playwright + page objects<br/>'does the flow work in a browser?'"']
        HD["BE: httptest handlers<br/>'right status, body, headers?'"']
    end
    subgraph L1["Layer 1 – Unit · offline · free"]
        VU["FE: Vitest + RTL<br/>'does this component render?'"']
        GU["BE: pkg/* packages<br/>'is this function correct?'"']
    end

    L1 --> L2 --> L3 --> L4

    classDef cheap fill:#1e4620,stroke:#5cb85c,color:#fff
    classDef mid fill:#1e3a5f,stroke:#4a90d9,color:#fff
    classDef costly fill:#5f1e1e,stroke:#d9534a,color:#fff
    class L1,L2 cheap
    class L3 mid
    class L4 costly

```

Layer	What it covers	How	A failure means
1 Unit	Pure logic: MDX loading, prompt building, config defaults, single components	Vitest + Testing Library / <code>go test</code>	The unit is wrong. Cheapest fix.
2 Component	A flow in a real browser; a handler over real HTTP	Playwright + page objects / <code>httptest</code>	Wiring between correct units broke.
3 Contract	Every documented response shape and failure mode at the <code>/api/chat</code> boundary	Mocked at the network seam	The producer changed, or we mis-handle a shape it always sent.
4 Live	Production is reachable and behaves; the model still answers accurately	Real network, real deploy, real model	Something outside the code changed.

****What is deliberately *not* covered:**** visual regression, cross-browser (chromium only), load/performance, and accessibility auditing beyond the role-based selectors the tests already depend on.

The rule that makes the split work

Layers 1–3 must run with no network and no API key. That is what keeps them usable as a pre-push gate. Both repos let you prove it rather than trust it:

```
env -u OPENAI_API_KEY go test ./...    # backend: no key, still green
SMOKE_BASE_URL= npm run e2e            # frontend: mocked at page.route
```

Layer 4 is the opposite by design: it exists precisely to touch the real thing, so it never runs as a silent gate.

Coverage thresholds

The frontend enforces **80% lines/statements/functions, 70% branches** on `lib/**` and components (`vitest.config.mts`). These are floors that fail the build, not targets to admire.

⚠ A glob gotcha worth knowing: the route group `(site)` cannot appear literally in a coverage include pattern — Vitest's matcher treats the parentheses as a group and silently drops every matching file, reporting a confident number for a fraction of the code. Use `app/**/components/**`.

3. Test seams

Each repo isolates the expensive dependency at exactly one boundary.

```
flowchart LR
    subgraph FEM["Frontend seam – browser network layer"]
        SPEC["spec"] --> FIX["chatApi fixture<br/>(fixtures.ts)"]
        FIX -->|page.route| INT["intercepted<br/>*/api/chat"]
    end
    end
    subgraph BEM["Backend seams – package-level func vars"]
        T1["api/chat_test.go"] -->|swap| V1["var callResponses"]
        T2["client_test.go"] -->|swap| V2["var sendResponse<br/>(SDK boundary)"]
        T3["handlers_test.go"] -->|APIKey:&quot;&quot;| V3["deterministic 502<br/>no stub needed"]
    end
    end

    classDef fe fill:#1e3a5f,stroke:#4a90d9,color:#fff
    classDef be fill:#1e4620,stroke:#5cb85c,color:#fff
    class SPEC,FIX,INT fe
    class T1,T2,T3,V1,V2,V3 be
```

Frontend — all mocking lives in the `chatApi` fixture; a spec that calls `page.route()` directly fails review. The fixture is *lazy*: importing it installs nothing, so smoke specs that never touch it intercept nothing.

Backend — no mocking library at all. Three hand-rolled seams: `callResponses` (handler level), `sendResponse` (SDK level), and the elegant one — passing `APIKey: ""` produces a real 502 through real code with zero network, because `CallResponses` guards on the empty key before constructing a client.

4. The wire contract

This is the highest-risk seam in the system: two independently deployed services, no shared types, no generated client.

```
sequenceDiagram
    participant B as Browser
    participant N as /api/chat (Next)
    participant G as Go handler
    participant O as OpenAI

    B->>N: POST {message}
    N->>G: POST {message}
    alt easter egg
        G-->>N: 200 {"output": "♪ Dookie..."}
    else normal
        G-->>O: Responses API (+ embedded context)
        O-->>G: output text
        G-->>N: 200 {"output": "..."}
    end
    N-->>B: pass through (JSON or text)

    Note over G,N: Errors are NOT JSON
    G-->>N: 405 empty body, no Content-Type
    G-->>N: 400 text/plain "invalid json: ..."
    G-->>N: 502 text/plain "openai error: ..."
```

The contract every consumer must handle:

Case	Status	Content-Type	Body
success	200	application/json	{"output":"..."}
non-POST	405	(none)	empty
malformed JSON	400	text/plain; charset=utf-8	invalid json: ...
upstream failure	502	text/plain; charset=utf-8	openai error: ...

Only the 200 is JSON. Calling `res.json()` without checking status throws on three of the four rows — which is why the frontend's error handling is contract-tested rather than assumed. To re-derive this table yourself:

```
curl -i -X POST https://stipes-openai-chat.vercel.app/chat \
  -H 'Content-Type: application/json' -d '{"message":"hello"}'
```

The backend's entire success shape is **one field**:

```
type ChatResponse struct {
    Output string `json:"output"`
}
```

Where the two repos disagree

The frontend parses responses through an 8-key fallback chain (`ChatWindow.tsx:62-73`):

```
choices[0].message.content → message.content → answer → reply → response → message →
text → output .
```

The backend only ever emits `output` — the 7th key. The first six are dead paths against this producer. That is defensible (the chain predates the Go service and tolerates a provider swap), but it should be a deliberate choice, not an accident:

- `docs/CRITICAL_FLOWS.md` documents the chain as a P0 invariant, and `chat-errors.spec.ts` has a passing test for **every** key — so the dead paths are locked in by tests that will never fail.
- The genuinely load-bearing behaviors are the **non-JSON error paths**, which

both repos do test: the Go side in `contract_test.go`, the Next side in `chat-errors.spec.ts` (500 → error bubble, abort → error bubble).

Recommendation: keep the chain, but annotate it — the comment should say *"the current producer returns `output` ; the other keys tolerate a provider swap"* so nobody "simplifies" it by deleting the one branch that fires.

Parity testing

`TestContract_ChatParityWithVercel` boots **both** Go implementations (`internal/handlers` and `api/chat.go`) side by side and asserts identical status, Content-Type, and body. Its own doc comment is admirably honest about the limit: both fail at the same empty-key check, so *prompt-level* drift is invisible to it. That class of bug was fixed structurally instead — prompt policy now lives only in `pkg/openai` , so the handlers cannot diverge.

5. Nondeterminism: how each side handles it

The two repos face the same problem — an LLM in the loop — and solve it differently, appropriately.

```
flowchart TB
    subgraph FEN["Frontend: avoid asserting on model output"]
        F1["smoke: assert SHAPE only<br/>bubble exists, non-empty, not error"]
        F2["chat-errors: mock the response<br/>exact assertions safe"]
    end
    subgraph BEN["Backend: grade the model statistically"]
        B1["RetryPolicy N=3 K=2<br/>sample 3x, need 2 passes"]
        B2["Normalize()<br/>fold quotes/dashes/markdown"]
        B3["Deterministic substring<br/>+ LLM judge (temp 0, score 1-5)"]
        B4["Corpus floor 0.90<br/>guardrail breach = hard fail"]
        B5["Replay goldens<br/>prompt_hash gates staleness"]
    end
    F1 -.same problem.- B1
    B1 --> B3 --> B4
    B2 --> B3

    classDef fe fill:#1e3a5f,stroke:#4a90d9,color:#fff
    classDef be fill:#1e4620,stroke:#5cb85c,color:#fff
    class F1,F2 fe
    class B1,B2,B3,B4,B5 be
```

Frontend rule: never assert exact text on anything live. The smoke chat test asserts a third bubble appeared, is non-empty, and is not the red error bubble — never *what* it says.

Backend rule: treat grading as sampling. Three details worth stealing:

1. **Transport errors are never grading failures.** A rate limit is recorded in

`Result.Err` , not as a hallucination.

1. **Short-circuit once the verdict is decided** — stop sampling when K passes

are banked or become unreachable.

- 1. `prompt_hash` **gates replay goldens**. Recordings carry a fingerprint of the prompt; when it drifts, replay tests *skip with a re-record instruction* rather than failing the build.

6. Using the Claude skills to speed up testing

Three skills automate the expensive, repetitive parts of this strategy. They are **procedures, not code generators with opinions of their own** — each reads this repo's conventions and the flows doc, so output matches what a reviewer here would expect.

Skill	Lives in	Turns this...	...into this
<code>/document-flows</code>	<code>~/Documents/claude-toolkit</code> (shared)	source + a deployed URL	<code>CRITICAL_FLOWS.md</code> + PDF + <code>@smoke</code> specs
<code>/playwright-tests</code>	<code>~/Documents/claude-toolkit</code> (shared)	the flows doc	page objects, fixtures, contract specs
<code>/contract-scan</code>	backend repo (<code>.claude/</code>)	a consumer repo	contract tests in <i>its</i> framework + defect report

The first two are symlinked into `~/ .claude/skills/`, so they load in any repo. `/contract-scan` is project-local to the backend and runs from there.

When to reach for which

Situation	Command	Run from
New to the repo, need to know what matters	<code>/document-flows</code>	either repo
Shipped a feature that adds a user-facing flow	<code>/document-flows</code> then <code>/playwright-tests <flow></code>	frontend
Changed a component's markup, labels, or roles	<code>/playwright-tests <flow></code>	frontend
Just deployed	<code>npm run smoke</code> (specs already generated)	frontend
Backend changed its response shape or status codes	<code>/contract-scan <consumer></code>	backend
Onboarding a <i>new</i> consumer of the API	<code>/contract-scan <repo></code>	backend
Changed the prompt or embedded documents	<code>make eval</code>	backend
Suspect docs have drifted from reality	<code>/document-flows <url></code>	either repo

****When *not* to:**** a one-line copy change, a dependency bump, or anything where you already know the single assertion you need. Writing one test by hand is faster than reviewing a generated suite.

The loop

```
flowchart LR
    SRC["source code<br/>+ deployed URL"] -->|"/document-flows"| DOC["CRITICAL_FLOWS.md<br/>invar"]
    DOC -->|"/playwright-tests"| SUITE["page objects<br/>fixtures<br/>specs"]
    SUITE --> RUN["npm run test<br/>npx playwright test"]
    RUN -->|"drift found"| DOC
    BE["backend repo"] -->|"/contract-scan"| CT["consumer contract tests<br/>+ CONTRACT-DEFECTS.m"]
    CT --> RUN

    classDef doc fill:#1e3a5f,stroke:#4a90d9,color:#fff
    classDef gen fill:#1e4620,stroke:#5cb85c,color:#fff
    classDef run fill:#4a3520,stroke:#d9a34a,color:#fff
    class DOC,CT doc
    class SUITE gen
    class RUN run
```

The flows doc is the spec. `/playwright-tests` refuses to run without it — that dependency is deliberate: tests generated from a reviewed inventory of invariants beat tests generated from a model's guess about what matters.

`/document-flows [url]` — write the spec

```
/document-flows                                # source-only
/document-flows https://www.stipes.tech        # + live verification
```

What a tester gets:

- **An invariant inventory.** Not "the chat works" but "the launcher keeps

`aria-label="Open Stipes bot"` because tests select on it" and *"do not narrow the response fallback chain — the upstream contract is loose."*

- **Criticality ranking** (P0/P1/P2) so smoke coverage goes where failure hurts.
- **A PDF** for people who don't read repos.
- **@smoke specs** wired to `SMOKE_BASE_URL`.
- **Drift notes.** With a URL, it exercises each flow in a real browser and

records where production disagrees with source rather than silently reconciling — that's how the duplicate-
<h1> defect surfaced.

Re-running is idempotent; content between `<!-- manual -->` markers survives.

`/playwright-tests [flow|all]` — generate the suite

```
/playwright-tests      # whole suite
/playwright-tests chat  # one flow
```

The parts a tester would otherwise hand-write:

1. **Page objects** with locators verified against a live accessibility tree

(Playwright MCP if configured, otherwise `claude-in-chrome`, otherwise a throwaway headless spec asserting `toHaveCount(1)`).

1. **Typed fixtures** — one per page object, plus a lazy network mock per loose

upstream contract. Lazy matters: importing the fixture installs no routes, so smoke specs can't accidentally mock.

1. **Error-contract specs** — the tedious part. One test per documented

response shape, per failure mode, per input guard. That's where `chat-errors.spec.ts` came from — one case per fallback key, plus the failure modes.

1. **A best-practices checklist** with grep-able rules (no `waitForTimeout`, no

`data-testid`, no `page.route` outside fixtures, no locators built in specs).

It also reports a **coverage map** — every invariant mapped to a test, with `GAP` rows for anything unmapped. That map is the review artifact, more useful than the diff.

`/contract-scan <repo>` — cross-repo defects

Run from the **backend** repo against a consumer:

```
/contract-scan ~/Documents/stipes-nextjs-tailwind
/contract-scan owner/repo --report-only
```

It derives the live contract from Go source (not the possibly-stale `openapi.yaml`), inventories every consumer call site, generates contract tests in the consumer's own framework (MSW / Vitest / Jest), and reports against an 11-item checklist. Two entries describe this pair exactly:

- **C1 (high) — unguarded response parsing:** the backend's 400/502 are

`text/plain` and 405 is empty, so an unconditional `res.json()` throws.

- **C8 (low) — phantom fields:** the consumer reads fields the backend never

emits. Our 8-key fallback chain, 7 of which are dead (see §4).

Guardrails worth knowing: it never edits consumer source (real bugs become skipped tests plus report entries), never pushes without confirmation, and never emits the string `easter` — which would trip the backend short-circuit and silently pass a test that never reached the model.

Where the time actually goes

Task	By hand	With skills	What still needs you
Map flows + invariants	hours of reading	minutes	Judging which invariants are real
Verify locators live	manual devtools	automatic	Confirming a11y gaps worth fixing
Error-contract matrix	tedious, skipped in practice	generated	Deciding which shapes exist
Cross-repo contract	rarely done at all	generated + report	Triage of defects found
Keeping docs current	rots immediately	re-run the skill	Reviewing the drift notes

What they do not replace: deciding what matters. A skill will happily generate a passing test for a dead code path — as ours did for all seven unused fallback keys. The generated coverage map, defect report, and drift notes are review artifacts, not verdicts.

Anti-pattern: running `/playwright-tests` and merging green. Read the coverage map's `GAP` rows and ask whether the invariants themselves are right. The suite is only as good as the flows doc it was generated from.

7. What runs when

```
flowchart LR
    DEV["local change"] --> FT["FE: npm run test<br/>+ npx playwright test"]
    DEV --> BT["BE: make test"]
    FT --> PUSH["push"]
    BT --> PUSH
    PUSH --> VD["Vercel auto-deploy<br/>(both repos)"]
    VD --> SMOKE["FE: npm run smoke<br/>vs www.stipes.tech"]
    VD -.deliberate.-> EVAL["BE: make eval<br/>~$0.03, prompt/doc changes"]

    classDef gate fill:#1e4620,stroke:#5cb85c,color:#fff
    classDef manual fill:#5f1e1e,stroke:#d9534a,color:#fff
    class FT,BT gate
    class SMOKE,EVAL manual
```

Command	Repo	Scope	Needs
<code>npm run test</code>	FE	Unit + coverage thresholds	nothing
<code>npx playwright test</code>	FE	All e2e tiers against a local server	nothing
<code>npm run smoke</code>	FE	<code>@smoke</code> only, against a deployment	<code>SMOKE_BASE_URL</code>
<code>SMOKE_BASE_URL= npm run e2e</code>	FE	Force a full local run despite <code>.env.local</code>	nothing
<code>make test</code>	BE	All free tests	nothing
<code>make eval</code>	BE	Live corpus + LLM judge	API key, ~cents/run
<code>make eval-record</code>	BE	Refresh replay goldens after a prompt change	API key

`SMOKE_BASE_URL` is the frontend's deploy-target switch: it swaps `baseUrl`, disables the local dev server, raises timeouts, adds retries, **and filters the run to `@smoke` only** — so the mocked tiers can never accidentally fire at production, even with the variable persisted in `.env.local`.

8. Structural gaps

Gaps in the *strategy* — things no amount of running the existing suites will catch. Individual open defects live in [CRITICAL_FLOWS.md](#), not here.

Gap	Where	Why it matters
No CI in either repo	both	Every gate is a local convention. Nothing stops a push that skipped the suites.
No cross-repo contract test	both	Nothing fails when the Go response shape changes. The frontend's fallback chain masks it until a reply renders as raw JSON.
No visual or cross-browser coverage	FE	Chromium only, no screenshot diffing. Layout and Safari-specific breakage is invisible.
No load or performance testing	both	The 60s upstream timeout and serverless cold starts are untested under concurrency.
No request logs in prod	BE	<code>LogReq</code> is wired only in <code>cmd/server</code> ; the Vercel functions the frontend actually calls have no middleware.
No metrics, tracing, or error reporting	both	Diagnosis relies on Vercel platform logs. No token/spend tracking.
<code>/healthz</code> doesn't check dependencies	BE	Returns <code>{"ok":true}</code> without validating the OpenAI key, so a service that 502s every request reports healthy.
Raw upstream errors echoed to client	BE	<code>fmt.Fprintf(w, "openai error: %v", err)</code> — minor information disclosure.

The one worth fixing first

A consumer-driven contract test. It is the only gap that hides a defect class rather than a category of test: the two services can drift apart silently and every suite stays green.

The tooling already exists — `/contract-scan` from the backend repo ([§6](#)). It closes the loop that `TestContract_ChatParityWithVercel` explicitly cannot reach across repo boundaries, and flags the phantom-field problem from §4 automatically. One command; the cost is triaging the output, not generating it.

After that, **CI** — because every layer described here is currently a convention rather than a gate.

Notes

Content between the manual markers is preserved verbatim when this document is regenerated. Put human-authored context here.